MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software and Automation Engineering



Report

on Laboratory Work No. 4

at **Operating Systems**

Topic: *The xv6 Shell from the Outside: Comparative Analysis of Minimalist Kernel Userland vs. GNU Bash Architecture*

Done by:
Student of gr. FAF-241
Tihon Aurelian-Mihai

Verified by:
univ. assist.
Reiţman Patricia

Chişinău – 2026

Contents

INTRODUCTION **2**

1 THEORETICAL FOUNDATIONS OF KERNEL USERLAND AND SHELL PARADIGMS **2**

1.1 Micro-Shell Versus Macro-Shell Architectural Models 3

1.2 Privilege Boundaries and System Call Trapping Mechanisms 3

1.3 Command Tree Representation and Recursive Execution 4

1.4 Pathname Resolution and the Environment Variable Void 4

1.5 Division of Responsibilities in Argument Tokenization 5

2 EMPIRICAL INVESTIGATION AND COMPARATIVE SYSTEM WALKTHROUGH **5**

2.1 Kernel Toolchain Inspection and sleep Utility Implementation 6

2.2 Exhaustive Feature Comparison: GNU Bash Versus xv6 Shell 7

2.3 Argument Reflection and Token Expansion Decomposition 10

2.4 Source Code Deconstruction of the xv6 Shell 12

CONCLUSIONS **14**

REFERENCES **15**

ANNEX A: SOURCE CODE LISTINGS **16**

INTRODUCTION

Operating systems userlands range from minimalist, bare-metal interactive consoles to rich, highly automated shell ecosystems. While contemporary Linux workstations rely on GNU Bash—a multifaceted language interpreter spanning over one hundred thousand lines of source code—educational operating systems such as MIT xv6 illustrate that the core user interface of a Unix-like system can be articulated entirely within a single program approximately five hundred lines in length. By contrasting these two execution environments side by side, systems engineers can distinguish between features fundamentally required by an operating system and convenience abstractions layered on top by modern userspace tools.

In xv6, the primary user interface resides in `user/sh.c`. This compact program translates keyboard keystrokes into kernel system calls, establishes inter-process pipelines, redirects standard input and output streams, and schedules background jobs. However, its deliberate structural minimalism omits ubiquitous modern amenities such as environment variable resolution, globbing expansions, history navigation, and programmable completion. Investigating how these omissions influence command interpretation reveals the strict boundary separating shell tokenization from kernel process execution.

The objective of this laboratory work is to dissect the xv6 shell from the outside, analyzing its structural capabilities and architectural limits through rigorous empirical comparison against GNU Bash. The study begins by verifying kernel toolchain compatibility and synchronizing the custom `sleep` system utility. Next, a systematic ten-point comparative benchmark is conducted across both shells to document operational divergences in redirection, pipelining, subshell grouping, and path lookup. Subsequently, a diagnostic argument reflection utility is implemented to establish conclusively whether wildcard expansions and quote removal are executed by the application binary, the operating system kernel, or the shell parser. Finally, the source code of `user/sh.c` is examined to deconstruct its internal abstract syntax tree and evaluate the parent-process mechanics of directory navigation. All source implementations and evaluation assets are hosted on GitHub and available via `https://github.com/aurelian-th/os-2026-TihonAurelian`.

1 THEORETICAL FOUNDATIONS OF KERNEL USERLAND AND SHELL PARADIGMS

1.1 Micro-Shell Versus Macro-Shell Architectural Models

The Unix operating system architecture is organized around a strict privilege dichotomy: the privileged supervisor kernel and unprivileged user space. Within user space, the shell occupies a unique role as the initial interactive process, but it enjoys no special hardware privileges. It interacts with the kernel exclusively through the standardized system call interface.

The design spectrum of Unix command interpreters can be categorized into micro-shells and macro-shells. A micro-shell, exemplified by xv6's `sh.c`, implements only the minimal grammar required to exploit core operating system primitives: process instantiation (`fork`), image replacement (`exec`), input/output diversion (`open`, `dup`, `close`), pipeline assembly (`pipe`), and exit synchronization (`wait`). It features no internal configuration files, no variable tables, and no terminal line-editing buffers.

In contrast, a macro-shell such as GNU Bash incorporates an extensive syntactic grammar, complex data structures, arithmetic evaluators, and terminal management engines. Bash maintains thousands of lines of terminal abstraction code via the GNU Readline library to process ANSI escape sequences, manage dynamic command history, and perform context-sensitive word completion. Understanding this distinction clarifies that modern command conveniences are not operating system kernel features, but rather user-space software engineering solutions built atop identical primitive system calls.

1.2 Privilege Boundaries and System Call Trapping Mechanisms

To maintain hardware isolation, modern processors enforce multiple execution modes. In the 64-bit RISC-V architecture targeted by xv6, execution is divided between User Mode (U-mode) and Supervisor Mode (S-mode).

When an unprivileged utility executes, the CPU core operates in U-mode, where access to privileged control and status registers (CSRs), page table base registers (`satp`), and direct physical hardware memory addresses is disabled by hardware logic. When the program requires kernel intervention—such as writing to a console or requesting timer delays—it loads the desired system call identifier into CPU register `a7`, places parameters into registers `a0` through `a5`, and executes the `ecall` (environment call) assembly instruction.

The `ecall` instruction triggers an intentional hardware trap. The processor hardware atomically records the program counter in the supervisor exception program counter (`sepc`), stores the trap cause in `scause`, switches the privilege level to S-mode, and vectors execution to the kernel trap handler desig-

nated by stvec. In xv6, the trap handler preserves user CPU registers into the process trapframe, extracts the system call number, and dispatches the request to the corresponding internal handler function in kernel/sysfile.c or kernel/sysproc.c. Once completed, the return value is placed in the trapframe's a0 slot, and the kernel executes the sret instruction to resume user-space execution.

1.3 Command Tree Representation and Recursive Execution

The internal parsing engine of user/sh.c constructs a recursive Abstract Syntax Tree (AST) representing the command line structure. Command structures share a common integer type tag, enabling polymorphic tree traversal in C through structured type casting.

Table 1.1 – xv6 Shell Abstract Syntax Tree Node Classifications

Macro	C Structure	Syntax Token	Structural Function
EXEC	struct execcmd	None (Default)	Leaf node holding argument vector argv[]
REDIR	struct redircmd	<, >, +	Decorator node specifying file and redirection mode
PIPE	struct pipecmd	|	Composite binary node linking left and right subtrees
LIST	struct listcmd	;	Sequential composite node executing subtrees in series
BACK	struct backcmd	&	Wrapper node delegating subtree to a background task

As summarized in Table 1.1, execution proceeds via the recursive function runcmd(struct cmd *cmd). When evaluating an EXEC node, the shell invokes exec(ecmd->argv[0], ecmd->argv). For a REDIR node, the target file descriptor is closed, the destination file is opened, and runcmd() is invoked recursively on the underlying command. For a PIPE node, pipe() allocates two descriptors, fork1() creates two child processes binding standard output and standard input to the pipe ends, and the parent waits for both sub-commands to finish.

1.4 Pathname Resolution and the Environment Variable Void

A fundamental divergence between xv6 and production Unix kernels lies in pathname resolution and the environment block. In Linux, the C runtime passes a third pointer array to main(): the environment array char *envp[]. When a relative command such as ls is executed, the shell searches each directory listed in the colon-separated PATH environment string.

In xv6, there are no environment variables and no global PATH search mechanism. The exec(path, argv) system call directly invokes the filesystem name lookup function namei(path). If path does not contain an absolute prefix, namei() searches exclusively within the current directory referenced by the calling process's cwd inode. If the requested binary is not located in the current working directory, exec() returns -1, and the shell prints an error diagnostic.

1.5 Division of Responsibilities in Argument Tokenization

A common misconception among application programmers is that user-space programs expand wildcard expressions or strip enclosing quotation marks. In reality, the POSIX execution contract delegates lexical tokenization strictly to user-space shells before the kernel is invoked.

When an operating system kernel executes `execve(path, argv, envp)`, it treats `argv` as an immutable array of null-terminated byte strings. The kernel allocates memory in the newly created user stack, copies each string pointer verbatim into the stack frame, sets the CPU register `a0` to `argc` and `a1` to `argv`, and jumps to the binary's entry point. The kernel performs zero string interpretation, zero quote stripping, and zero pattern matching. Consequently, if a shell lacks wildcard expansion algorithms—as is the case with xv6—wildcard characters remain unexpanded and are delivered directly to the receiving program as literal bytes.

2 EMPIRICAL INVESTIGATION AND COMPARATIVE SYSTEM WALKTHROUGH

2.1 Kernel Toolchain Inspection and sleep Utility Implementation

The practical investigation commenced by validating the system call signatures in the xv6 source tree and ensuring toolchain compatibility. Upstream modifications in August 2025 altered system call naming in certain xv6 branches, renaming timer delays from `sleep()` to `pause()`.

```
phantom@0xAu: ~/xv6-riscv

phantom@0xAu:~/xv6-riscv$ cd ~/xv6-riscv
phantom@0xAu:~/xv6-riscv$ grep -n "pause\|sleep" user/user.h
25:int sleep(int);
```

Figure 2.1 – Inspection of system call prototypes in user/user.h

According to Figure 2.1, executing `grep -n "pause\|sleep" user/user.h` confirms that line 25 of the active kernel tree declares `int sleep(int)`. Therefore, the timer system call in this repository retains the canonical identifier `sleep()`.

The user-space utility `user/sleep.c` was authored to interface with this system call, providing argument checking and error handling.

```
phantom@0xAu: ~/xv6-riscv

phantom@0xAu:~/xv6-riscv$ cat user/sleep.c
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  if(argc != 2){
    fprintf(2, "usage: sleep <ticks>\n");
    exit(1);
  }
  sleep(atoi(argv[1]));
  exit(0);
}
```

Figure 2.2 – Source implementation of the xv6 sleep user utility

As shown in Figure 2.2, the program validates that exactly one argument is supplied (`argc == 2`). If the argument count is invalid, it writes a usage diagnostic to file descriptor 2 via `fprintf(2, "usage: sleep <ticks>\n")` and terminates with status 1. If valid, it converts the ASCII string to an integer using `atoi(argv[1])` and invokes the `sleep()` system call, yielding the CPU for the designated duration.

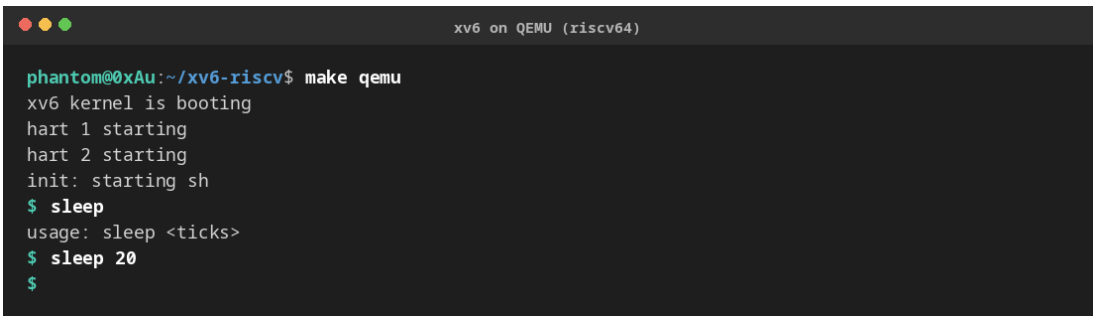


Figure 2.3 – Compilation and execution testing of sleep inside xv6 QEMU

The program was registered in the `Makefile` by appending `$U/_sleep\` to the `UPROGS` rule. Figure 2.3 illustrates the boot sequence under QEMU RISC-V 64. Invoking `sleep` without parameters triggers the error guard, printing the usage prompt. Providing a numeric parameter (`sleep 20`) pauses execution for 20 clock ticks and returns cleanly to the shell prompt.

2.2 Exhaustive Feature Comparison: GNU Bash Versus xv6 Shell

To evaluate architectural differences between a modern production shell and a minimalist educational userland, a systematic benchmark of ten core command features was executed across both environments. The Linux baseline was evaluated within a dedicated sandbox directory containing an initial file named `README`.

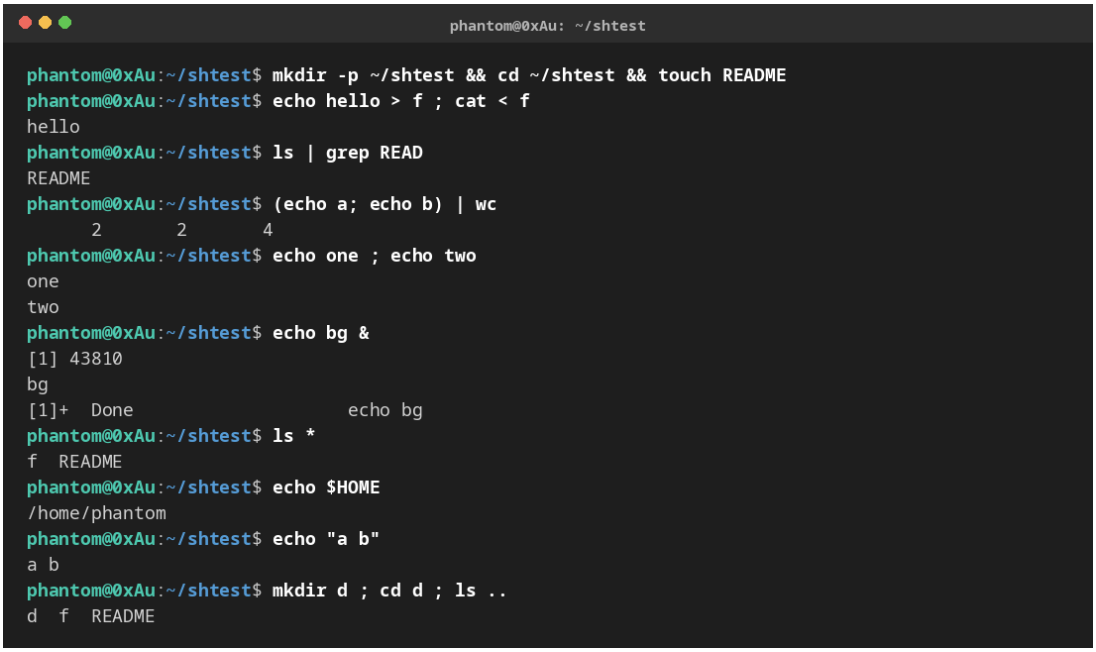


Figure 2.4 – Execution of the comparative command suite in GNU Bash

The identical command suite was subsequently submitted to the xv6 interactive shell under QEMU emulation.



Figure 2.5 – Execution of the comparative command suite in the xv6 shell

A systematic breakdown of the observable outcomes and underlying architectural explanations is synthesized in Table 2.1.

Table 2.1 – Comparative Architectural Matrix: GNU Bash Versus xv6 Shell

Command Tested	Feature	GNU Bash Output	xv6 Shell Output	Architectural Analysis
`echo hello > f ; ...`	Redirection	`hello`	`hello`	Both shells support < and > descriptor duplication.
`ls | grep READ`	Pipe	`README`	`README 2 2 2292`	Both shells construct unidi tional kernel pipes via `pipe(`
`(echo a; echo b) | wc`	Subshell	`2 2 4`	Syntax error	xv6 parser lacks grammar s port for parenthetical subshel
`echo one ; echo two`	Sequence	`one / two`	`one / two`	Both shells implement sequer execution via `LIST` nodes.
`echo bg &`	Background	`[1] 43810 bg`	`bg`	Both support `&`; Bash reports ID and process monitoring.
`ls *`	Globbing	Lists matching files	`cannot access *`	xv6 lacks pathname pat matching; passes literal asteri
`echo $HOME`	Variables	`/home/phantom`	`$HOME`	xv6 contains no environm variable storage or expansion
`echo "a b"`	Quotes	`a b`	`"a b"`	xv6 tokenizer preserves quote literal characters.
`mkdir d ; cd d ; ls ..`	Path lookup	Lists parent directory	`exec ls failed`	xv6 lacks `PATH`; searches c within current directory.
`Up, Tab, Ctrl+R`	Line editing	History and completion	Literal escape codes	xv6 console driver reads bytes; lacks Readline engine.

The failure of `ls ..` following directory navigation inside xv6 exposes a critical architectural characteristic of minimalist systems.



Figure 2.6 – Directory navigation path failure and absolute path workaround

As demonstrated in Figure 2.6, executing `mkdir d ; cd d` transitions the shell into the subdirectory. However, subsequently executing `ls ..` fails immediately with `exec ls failed`. Because xv6 lacks a `PATH` environment variable, the `exec()` call attempts to resolve the relative binary name `ls` against the current directory inode (`d/ls`). Because user binaries reside in the root filesystem directory (`/`), the lookup fails. To execute the binary successfully from within a child directory, the user must supply the absolute path explicitly: `/ls ..`.

Synthesizing these observations addresses the core laboratory inquiries:

a) **Common Versus Exclusive Features:** Both shells share foundational Unix primitives: stream redirection (`>`, `<`), inter-process piping (`|`), sequential execution (`;`), and asynchronous background tasking (`&`). Conversely, features exclusive to Bash include pathname globbing (`*`), parameter expansion (`$`), quote processing, subshell grouping (`(...)`), `PATH` directory resolution, and interactive line editing.

b) **Navigation Failure Remediation:** The failure of `ls` inside `d` arises from the absence of search path resolution. The working alternative is to prefix the executable name with an absolute root slash (`/ls ..`), instructing the kernel filesystem layer to resolve the inode starting from the root directory rather than `cwd`.

c) **Utility Impact Assessment:** In daily engineering workflows, the omission of interactive line editing (command history and Tab completion) and `PATH` lookup represents the most severe productivity impediment. Retyping lengthy absolute paths and identical commands incurs substantial friction. Conversely, the easiest feature to incorporate into `user/sh.c` would be a hardcoded `PATH` fallback: if `exec(cmd, argv)` fails and `cmd` does not begin with a slash, prepend `"/"` and retry the execution once before reporting failure.

2.3 Argument Reflection and Token Expansion Decomposition

To prove definitively which software layer performs parameter expansion, whitespace tokenization, and quote removal, diagnostic argument reflection utilities were implemented for both platforms.

```
phantom@0xAu: ~/xv6-riscv

phantom@0xAu:~/xv6-riscv$ cat user/args.c
#include "kernel/types.h"
#include "user/user.h"

int main(int argc, char *argv[]) {
  printf("argc = %d\n", argc);
  for(int i = 0; i < argc; i++)
    printf("argv[%d] = [%s]\n", i, argv[i]);
  exit(0);
}
phantom@0xAu:~/xv6-riscv$ grep -n -C 2 "_args" Makefile
144-	$U/_forphan\
145-	$U/_dorphan\
146-	$U/_sleep\
147:	$U/_args\
148-
```

Figure 2.7 – Source implementation of user/args.c and Makefile configuration

Figure 2.7 details the source code of `user/args.c`. The program inspects `argc` and iterates through `argv`, printing each string enclosed in square brackets: `printf("argv[%d] = [%s]\n", i, argv[i])`. The utility was registered in the `Makefile` by inserting `$U/_args\` into `UPROGS`. An identical program, `args_linux.c`, was compiled on the host system using `gcc`.

The diagnostic programs were executed with identical argument patterns across both shells.

```
xv6 on QEMU (riscv64)

$ args one    two    three
argc = 4
argv[0] = [args]
argv[1] = [one]
argv[2] = [two]
argv[3] = [three]
$ args *.c "x y"
argc = 4
argv[0] = [args]
argv[1] = [*.c]
argv[2] = ["x]
argv[3] = [y"]
$ args $HOME
argc = 2
argv[0] = [args]
argv[1] = [$HOME]
$ args 'single quoted'
argc = 3
argv[0] = [args]
argv[1] = ['single]
argv[2] = [quoted']
```

Figure 2.8 – Argument reflection outputs inside the xv6 shell

Figure 2.8 illustrates argument handling inside xv6:

– `args one two three`: yields `argc = 4`. Intermediate spaces are discarded by whitespace delimiters.
– `args *.c "x y"`: yields `argc = 4`. Arguments are `argv[1] = [*.c]`, `argv[2] = ["x]`, and `argv[3] = [y"]`. The wildcard is unexpanded, and the quotation marks remain intact while the whitespace inside quotes causes an erroneous split.
– `args $HOME`: yields `argc = 2`, with `argv[1] = [$HOME]` preserved literally.
– `args 'single quoted'`: yields `argc = 3`, splitting into `['single]` and `[quoted']`.

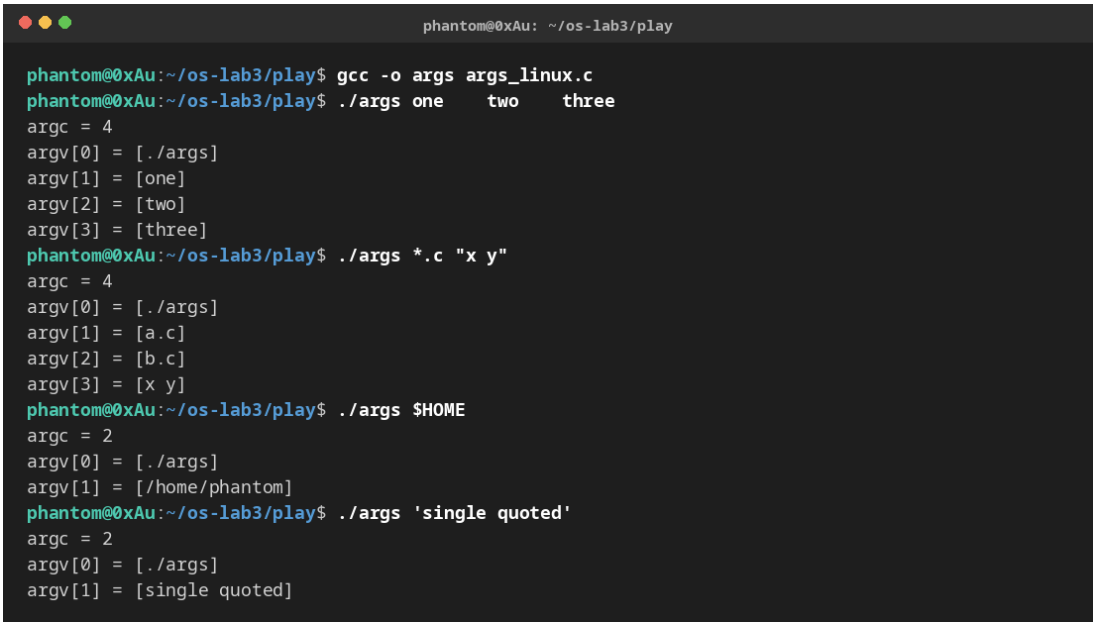


Figure 2.9 – Argument reflection outputs inside GNU Bash on Linux

Figure 2.9 displays the identical test evaluated within GNU Bash:

– `./args one two three`: yields `argc = 4`. Word splitting consolidates consecutive whitespace characters into single delimiters.
– `./args *.c "x y"`: yields `argc = 4`. The wildcard expands to matching files (`argv[1] = [a.c]`, `argv[2] = [b.c]`), while the double quotes prevent word splitting and are stripped, producing `argv[3] = [x y]`.
– `./args $HOME`: yields `argc = 2`, resolving to `argv[1] = [/home/phantom]`.
– `./args 'single quoted'`: yields `argc = 2`, stripping quotes to produce `argv[1] = [single quoted]`.

Comparing these outputs answers the experimental inquiries decisively:

a) **Argument Counts for Wildcard Inputs:** In Bash, `args *.c "x y"` yielded 4 arguments (`argv[0]` plus `a.c`, `b.c`, and `x y`). In xv6, the identical command also yielded 4 arguments, but with completely different content: `argv[0]` plus literal `*.c`, `"x`, and `y"`.
b) **Locus of Expansion and Quote Removal:** Pathname expansion and quote stripping are executed

exclusively by the **shell**. The receiving application program receives only the final string array, and the kernel merely copies pointers into the stack frame. Because xv6 contains no expansion routines, the literal characters pass unhindered to the program.

c) **Whitespace Consolidation:** In both shells, extra spaces between unquoted tokens are discarded during lexical scanning. The tokenizer treats consecutive whitespace characters as a single argument boundary.

2.4 Source Code Deconstruction of the xv6 Shell

The final phase examined the internal structure of `user/sh.c`, which totals 506 lines of C code.

```
phantom@0xAu: ~/xv6-riscv

phantom@0xAu:~/xv6-riscv$ wc -l user/sh.c
506 user/sh.c
phantom@0xAu:~/xv6-riscv$ grep -n "#define" user/sh.c
7:#define EXEC  1
8:#define REDIR 2
9:#define PIPE  3
10:#define LIST  4
11:#define BACK  5
13:#define MAXARGS 10
```

Figure 2.10 – Command type definitions in user/sh.c

Figure 2.10 inspects the macro definitions establishing the command vocabulary of xv6. Five distinct command kinds are defined:

- `EXEC` (1): Plain binary execution, triggered by default word tokens.
- `REDIR` (2): File stream redirection, triggered by <, >, and +.
- `PIPE` (3): Inter-process streaming, triggered by the vertical bar |.
- `LIST` (4): Sequential command execution, triggered by the semicolon ;.
- `BACK` (5): Asynchronous background execution, triggered by the ampersand &.

```
phantom@0xAu: ~/xv6-riscv

phantom@0xAu:~/xv6-riscv$ grep -n "case " user/sh.c
61:  case EXEC:
68:  case REDIR:
77:  case LIST:
84:  case PIPE:
103:  case BACK:
```

Figure 2.11 – Dispatch switch statements in the runcmd function

Figure 2.11 reveals the execution dispatcher in `runcmd()`. Beginning at line 61, a `switch` statement directs each command node to its specific execution handler. Redirections open target files and recurse; pipes allocate kernel channels and fork paired children; sequential lists execute left and right subtrees; and

background nodes spawn detached children.

```
phantom@0xAu: ~/xv6-riscv

phantom@0xAu:~/xv6-riscv$ sed -n '166,174p' user/sh.c
    if(cmd[0] == 'c' && cmd[1] == 'd' && cmd[2] == ' '){
      // Chdir must be called by the parent, not the child.
      cmd[strlen(cmd)-1] = 0;  // chop \n
      if(chdir(cmd+3) < 0)
        fprintf(2, "cannot cd %s\n", cmd+3);
    } else {
      if(fork1() == 0)
        runcmd(parsecmd(cmd));
      wait(0);
```

Figure 2.12 – Parent-process handling of directory changes in user/sh.c

Figure 2.12 inspects lines 166 through 174 of `user/sh.c`, which contain the explicit implementation of the `cd` command. The source code includes an explanatory comment: `// Chdir must be called by the parent, not the child.` Because the working directory is an internal attribute of the calling process, delegating `cd` to a child process created via `fork()` would alter only the child's working directory before termination. By inspecting for the `cd` prefix directly in `main()`, the shell intercepts the command before calling `fork()`, executing `chdir()` in the parent address space.

Finally, searching `user/sh.c` for occurrences of `*` or `$` yields zero matches. The lexical tokenizer in `gettoken()` recognizes only whitespace and the structural symbols `<|>;&()`. Because neither wildcard matching nor variable substitution algorithms exist within the parser, the shell treats these symbols as standard alphanumeric characters, directly explaining the empirical findings of Section 2.2.

CONCLUSIONS

This laboratory work completed an intensive architectural deconstruction of the xv6 operating system shell and conducted an empirical comparative analysis against GNU Bash. By contrasting a 500-line educational command interpreter with a production-grade POSIX shell, the investigation exposed the fundamental boundary separating core kernel abstractions from userspace software enhancements.

The experimental results demonstrated that the core Unix paradigm does not require massive codebases to achieve functional elegance. Using fewer than a dozen elementary system calls—specifically `fork`, `exec`, `pipe`, `open`, `close`, `dup`, and `wait`—the xv6 shell constructs arbitrary process pipelines, orchestrates input and output redirection, and manages concurrent background tasks. These features operate at near parity with modern Linux systems because they leverage direct kernel primitives rather than userspace emulation.

Conversely, the study illuminated the exact locus of command-line expansions. Through the diagnostic reflection utilities `args` and `args_linux`, it was definitively demonstrated that neither application binaries nor the operating system kernel possess any awareness of wildcards or quotation syntax. The operating system kernel treats argument vectors strictly as raw, contiguous byte arrays in memory. The expansion of asterisks into matching filenames and the removal of quotation marks are performed entirely within user space by the shell prior to invoking `execve()`. When using a minimalist shell that omits these tokenizer stages, special characters flow verbatim into the target program.

Finally, source code analysis of `user/sh.c` demystified the necessity of shell built-ins. The explicit examination of the `cd` implementation confirmed that operations altering process-level execution context—such as current directory inodes or environment tables—cannot be implemented as independent binary executables due to process memory isolation. Overall, this laboratory work provided essential systems programming insights, bridging the conceptual gap between high-level shell interaction and bare-metal kernel mechanics.

REFERENCES

[1] Cox, R., Kaashoek, F., and Morris, R. *xv6: a simple, Unix-like teaching operating system*. Cambridge, MA: MIT CSAIL, 2022.

[2] Silberschatz, A., Galvin, P. B., and Gagne, G. *Operating System Concepts*. 10th ed. Hoboken, NJ: John Wiley & Sons, 2018.

[3] Tanenbaum, A. S., and Bos, H. *Modern Operating Systems*. 4th ed. Boston, MA: Pearson, 2015.

[4] Kerrisk, M. *The Linux Programming Interface: A Linux and UNIX System Programming Handbook*. San Francisco, CA: No Starch Press, 2010.

[5] Love, R. *Linux System Programming: Talking Directly to the Kernel and C Library*. 2nd ed. Sebastopol, CA: O'Reilly Media, 2013.

ANNEX A: SOURCE CODE LISTINGS

A.1 Implementation of user/sleep.c in xv6

```c
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  if(argc != 2){
    fprintf(2, "usage: sleep <ticks>\n");
    exit(1);
  }
  sleep(atoi(argv[1]));
  exit(0);
}
```

A.2 Implementation of user/args.c in xv6

```c
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  printf("argc = %d\n", argc);
  for(int i = 0; i < argc; i++)
    printf("argv[%d] = [%s]\n", i, argv[i]);
  exit(0);
}
```

A.3 Implementation of args_linux.c for Linux

```c
#include <stdio.h>

int
main(int argc, char *argv[])
{
  printf("argc = %d\n", argc);
  for(int i = 0; i < argc; i++)
    printf("argv[%d] = [%s]\n", i, argv[i]);
  return 0;
}
```

A.4 Excerpt of UPROGS Registration in xv6 Makefile

```
UPROGS=\
    $U/_cat\
    $U/_echo\
    $U/_forktest\
    $U/_grep\
    $U/_init\
    $U/_kill\
    $U/_ln\
    $U/_ls\
    $U/_mkdir\
    $U/_rm\
    $U/_sh\
    $U/_stressfs\
    $U/_usertests\
    $U/_grind\
    $U/_wc\
    $U/_zombie\
    $U/_logstress\
    $U/_forphan\
    $U/_dorphan\
    $U/_sleep\
    $U/_args\
```